

VIBEATHON CÔTE D'IVOIRE 2026 · SECTEUR ÉNERGIE

EcoWatt

Un réseau de prises intelligentes piloté par IA qui mesure la consommation réelle, coupe le gaspillage et décale les usages, expliqué et prêt à connecter au matériel.

DOCUMENT

Guide complet

ÉQUIPE

Coding Team

LEAD TECHNIQUE

Kader

DATE

Juillet 2026

1 Le projet en bref

Aucun foyer ivoirien ordinaire n'a de compteur intelligent. EcoWatt en fabrique un, accessible : des prises qui mesurent chaque appareil, et une IA qui coupe le gaspillage et décale les usages hors des heures de pointe.

Le problème

- Les factures montent sans qu'on comprenne pourquoi.
- Zéro visibilité sur la consommation, appareil par appareil.
- Les pics en heure de pointe surchargent le réseau et provoquent des coupures.
- L'électricité venant surtout du gaz, chaque kWh gaspillé émet du CO2.

La solution

Chaque prise intelligente (ESP32 + capteur + relais) mesure la consommation réelle et peut couper l'appareil. Une IA reçoit les mesures, décide quoi couper ou décaler, commande la bonne prise, et explique chaque décision.

Comment l'IA décide (4 facteurs)

FACTEUR	RÔLE
Priorité de l'appareil	Essentiel (jamais coupé), reportable (décalable), confort.
Le moment	En heure de pointe, le réseau est chargé et l'électricité la plus polluante.
L'objectif	À l'approche d'un budget, on déleste les appareils reportables.
La conso réelle	Mesurée par la prise, en temps réel.

Les fonctionnalités livrées

● Tableau de bord temps réel

Conso instantanée, appareils actifs, énergie évitée, économies.
Graphe de consommation en direct.

● Commande vocale

Dictée au micro (Chrome/Edge) et lecture à voix haute des réponses (dont Firefox).

● Impact

kWh, FCFA et CO2 économisés, calculés sur la conso réellement évitée + projection nationale.

● Réseau

État du backend, du cerveau IA, du transport, et liste des prises connectées.

● Mode démo / réel

Un interrupteur bascule à chaud entre scénario simulé et données réelles du backend.

● Responsive

Du plein écran au mobile, avec sidebar en tiroir sur petit écran.

● Assistant IA conversationnel

Un vrai LLM (Llama 3.3 70B via Groq) qui répond à tout sur EcoWatt et exécute des ordres ("coupe la bouilloire").

● Décisions de l'IA

Historique de chaque coupure/rallumage, toujours justifié en langage clair.

● Historique

Évolution de la consommation et journal des événements.

● Réglages

Heure de pointe, seuil de pic, objectif d'économies, modèle IA.

● Mode sombre (noir pur)

Bouton dédié, tous les éléments gérés comme en clair, choix mémorisé.

● Auto-découverte

Aucun appareil pré-inscrit : dès qu'une prise se branche, elle apparaît toute seule.

3 Comment utiliser l'application

Lancer l'application (en local)

```
# Backend (API + IA)
cd backend
.venv/Scripts/python -m uvicorn app.main:app --host 0.0.0.0 --port 8000

# Frontend (dashboard)
cd app
npm run dev           # ouvre http://localhost:5173
```

Naviguer

La barre latérale gauche donne accès à toutes les pages : Tableau de bord, Assistant IA, Appareils, Décisions IA, Impact, Historique, Réseau, Réglages. Sur mobile, un bouton menu ouvre la barre en tiroir.

Mode démo ou réel

- **Mode démo** (par défaut) : l'app rejoue un scénario simulé, sans matériel. Idéal pour présenter.
- **Mode réel** : clique la carte "Mode démo" en bas de la barre pour basculer. Le dashboard se connecte au backend et affiche les vraies prises. Vide tant qu'aucun matériel n'est branché.

Parler à l'IA

- 1 Va sur **Assistant IA**.
- 2 Écris une question ("Quel appareil consomme le plus ?") ou un ordre ("Coupe la bouilloire").
- 3 Ou clique le **micro** (Chrome/Edge), parle, et l'IA n'envoie qu'à la fin de ta phrase.
- 4 Active **Lecture vocale** pour qu'elle lise ses réponses à voix haute.

Mode sombre

Le bouton soleil/lune en haut à droite bascule en noir pur. Le choix est mémorisé.

4

Architecture & technologies

Boîtier ESP32 (prise)

Mesure la puissance, coupe/rallume via relais.

↓ **WiFi (WebSocket) ou Bluetooth (BLE)**

Backend — Python / FastAPI + SQLite

Ingestion, cerveau IA, actionneur, calcul d'impact, historique.

↓ **WebSocket temps réel**

Dashboard — React / Vite / Tailwind

Conso live, décisions, impact, assistant IA.

COUCHE	TECHNOLOGIE
Prises / firmware	ESP32 + capteur de courant + relais (Arduino / PlatformIO)
Communication	WebSocket direct (WiFi) ou BLE (Bluetooth), sans broker
Backend	Python, FastAPI, SQLite
Application	React 19, Vite, TypeScript, Tailwind CSS
Cerveau IA	Llama 3.3 70B via Groq (gratuit), fournisseur interchangeable

5 Brancher le matériel — WiFi

Le boîtier et le PC (backend) doivent être sur le même réseau WiFi. Le boîtier vise l'IP locale du PC. Dès sa première mesure, l'appareil apparaît tout seul.

a. Matériel par prise

- **ESP32** (le cerveau du boîtier).
- **Capteur de courant** (ex. SCT-013 + circuit de mesure) : mesure la consommation.
- **Module relais** conçu pour le secteur : coupe/rallume l'appareil.

SÉCURITÉ 220V

On manipule du 220 volts, c'est dangereux. Modules secteur uniquement, jamais de fil nu sous tension, montage supervisé par quelqu'un qui s'y connaît.

b. Côté PC

- 1 PC et ESP32 sur le **même WiFi**.
- 2 Récupérer l'**IP locale du PC** (`ipconfig` → adresse IPv4 du Wi-Fi). Exemple actuel : `192.168.1.19` .
- 3 Lancer le backend en écoute réseau : `uvicorn app.main:app --host 0.0.0.0 --port 8000` .
- 4 Autoriser Python dans le **pare-feu Windows** (fenêtre au 1er lancement).

c. Configurer le firmware

Dans `firmware/src/main.cpp`, en haut :

```
WIFI_SSID      = "TON_WIFI";           // même WiFi que le PC
WIFI_PASS      = "TON_MOT_DE_PASSE";
BACKEND_HOST   = "192.168.1.19";       // IP locale du PC
BACKEND_PORT   = 8000;
PRISE_ID       = "p1";                 // unique par prise
DEVICE_ID      = "kettle-1";
DEVICE_NAME    = "Bouilloire";         // nom affiché
DEVICE_PRIORITE = "reportable";        // essentiel | reportable | confort
```

d. Flasher l'ESP32

```
cd firmware
pio run -t upload      # compile et téléverse (USB)
pio device monitor     # logs : [wifi] IP ... puis [ws] connecte au backend
```

e. Voir l'appareil apparaître

- 1 Dans l'app, passer en **Mode réel**.
- 2 Alimenter le boîtier.
- 3 À la première mesure, l'appareil **apparaît automatiquement** sur le dashboard, la conso monte, l'IA peut le piloter.

CALIBRATION (IMPORTANT)

Dans `readWatts()`, la constante `AMPS_PER_ADC` est à 0 par défaut. Pour des watts justes, branche une charge connue (ex. ampoule 60 W), relève la valeur du capteur, et ajuste ce facteur. Sans calibration, la prise remonte 0 W (mais connexion et relais marchent).

6 Brancher le matériel — Bluetooth (option)

Le WiFi reste recommandé (portée, always-on, multi-prises). Le Bluetooth est un plan B quand il n'y a pas de WiFi. Le cerveau (IA, impact, dashboard) est identique.

Côté PC

```
pip install bleak
set ECOWATT_BLE=1
.venv/Scripts/python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Côté boîtier

- Flasher `firmware/ble/main_ble.cpp` à la place du firmware WiFi.
- Mêmes réglages `PRISE_ID` / `DEVICE_ID` / `DEVICE_NAME` / `DEVICE_PRIORITE`.

Le boîtier annonce un nom `EcoWatt-...` ; le backend le détecte, s'abonne aux mesures et envoie les ordres. WiFi et Bluetooth peuvent coexister sur le même dashboard. Nécessite un adaptateur Bluetooth sur le PC.

7 Dépannage

SYMPTÔME	CAUSE PROBABLE / SOLUTION
"Connexion au backend..." reste affiché	Backend non lancé, ou pas en <code>--host 0.0.0.0</code> . Tester <code>http://IP_DU_PC:8000/docs</code> .
L'appareil n'apparaît pas	PC et ESP32 pas sur le même WiFi, mauvaise IP dans le firmware, ou pare-feu qui bloque le port 8000.
WebSocket se déconnecte en boucle	IP/port faux ou pare-feu. Vérifier les logs <code>pio device monitor</code> .
Conso à 0 W	Capteur non calibré (voir §5.e) ou relais coupé.
Le micro n'apparaît pas	Reconnaissance vocale non supportée par Firefox : utiliser Chrome ou Edge.
L'IA répond de façon générique	Quota gratuit atteint ou clé absente : l'app retombe sur des règles. Vérifier la clé et le fournisseur.